KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

함수 1
정의와 return

자주 쓰는 코드에 이름을 붙이고
값을 넣어 결과를 돌려받기

```
def calc_total(cart, menu):
    total = 0
    for m in cart:
        total += menu[m]
    return total
calc_total(cart, menu)
```

오늘의 순서

1 왜 필요한가

손님마다 복사한 계산, 재료와 결과 상자, 가장 짧은 def

4–6쪽

2 정의와 호출

def 줄 다섯 조각, 정의만 하면 안 찍혀요, 몸통으로 갔다 돌아오기

7–9쪽

3 매개변수와 인자

재료 칸 하나, 여러 개, 순서대로 들어가요

10–12쪽

4 return

돌려주기, print와 비교, None, 바로 끝내기, 값 여러 개

13–17쪽

5 오류와 함정

괄호 빼기, 인자 수 TypeError, return 아래 줄, None 계산, NameError

18–22쪽

6 실습

장바구니 총액, 품절 확인, 성적 등급, 할인 결제

23–26쪽

3쪽은 노트북 준비, 27–28쪽은 막힐 때와 한 장 요약이에요.

들어가기

18에서 이어서 해요

- hello-uv 노트북, 커널 .venv — 09 · 25–27쪽
- 변수 이름 규칙 — 10
- f-string, 리스트, for, 튜플 꺼내 담기 — 13
- 딕셔너리 menu["latte"], None — 15
- if, elif, else — 17
- range, while, break — 18

기억이 안 나면 오른쪽에 적힌 챕터를 먼저 다시 봐요.

노트북 다시 열기

1. VS Code로 hello-uv 폴더 열기
2. 탐색기에서 `basics.ipynb` 열기
3. 오른쪽 위 커널이 .venv인지 보기
4. 셀에 적고 `Shift` + `Enter` 로 실행

1. 왜 필요한가

같은 계산을 손님마다 복사하고 있어요

[1]

```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500}
cart = ["coffee", "cake"]
total = 0                                          # 첫 손님
for m in cart:
    total += menu[m]
print(f"합계 {total:,}원")
cart = ["latte", "latte", "cake"]
total = 0                                          # 똑같은 4줄
for m in cart:
    total += menu[m]
print(f"합계 {total:,}원")
```

Python

```
합계 11,000원
합계 16,500원
```

손님이 올 때마다 4줄 복사

세 번째 손님이 오면 또 붙여 넣어요. 노트북이 같은 줄로 금방 길어져요.

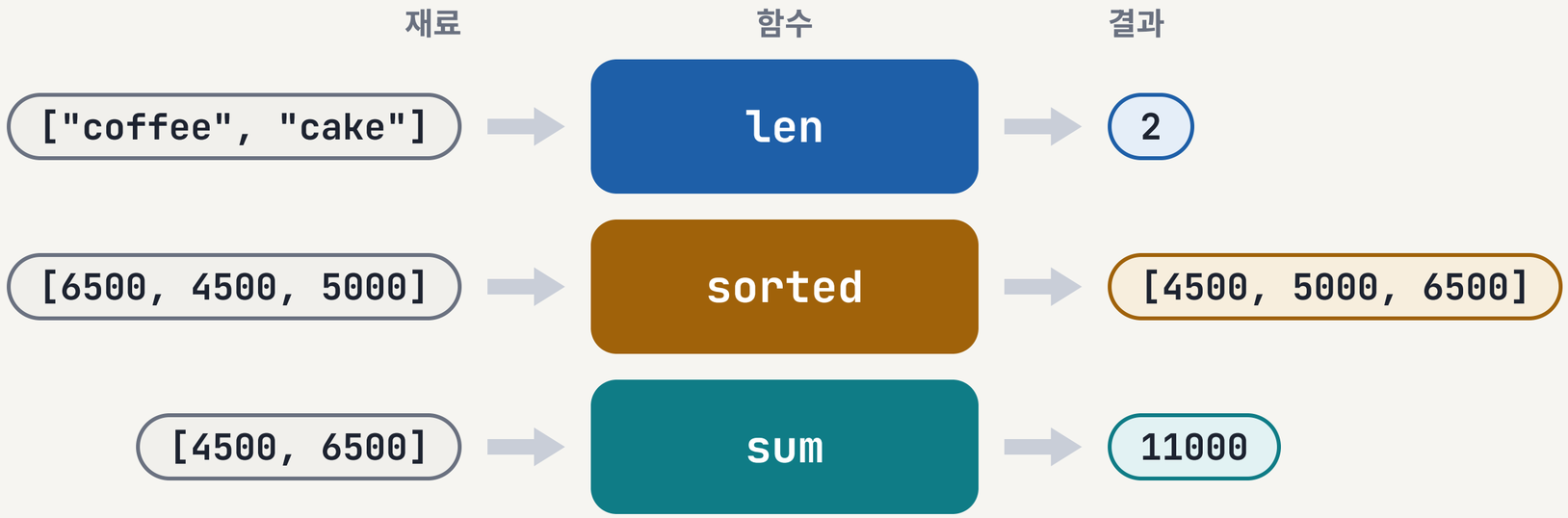
⚠ 고칠 때 한 곳을 빠뜨리면

할인 규칙이 생기면 복사한 곳을 전부 고쳐야 해요. 한 곳만 빠뜨려도 손님마다 금액이 달라져요.

같은 일을 하는 줄에 이름을 붙이고, 이름으로 불러 쓰는 게 함수예요.

1. 왜 필요한가

함수는 재료를 넣으면 결과가 나오는 상자예요



[2]
```python
cart = ["coffee", "cake"]
print(len(cart))
print(sorted([6500, 4500, 5000]))
print(sum([4500, 6500]))
```

```
2
[4500, 5000, 6500]
11000
```

13에서 이미 써 봤어요

len, sorted, sum은 파이썬에 들어 있는 함수예요. 오늘은 이런 상자를 def로 직접 만들어요.

1. 왜 필요한가

def로 만들고, 이름()으로 불러요

[3]

```python
def hello():                    # 만들기
    print("어서 오세요")

hello()                         # 부르기
hello()
```

```
어서 오세요
어서 오세요
```

def 함수를 만들어요. 정의라고 해요.

hello() 만든 함수를 실행해요. 호출이라고 해요.

부른 만큼 실행돼요

hello()를 두 번 불러서 "어서 오세요"가 두 번 찍혔어요. 몸통은 한 번만 적었어요.

만드는 걸 정의, 부르는 걸 호출이라고 해요.

2. 정의와 호출

def 줄은 다섯 조각으로 읽어요

```
def hello():
    print("어서 오세요")
```

이름은 하는 일로 지어요

hello, calc_total, print_menu처럼 소문자와 밑줄로 적어요. 이름만 봐도 무엇을 하는지 알 수 있게 동사를 넣으면 좋아요.

`def`	함수를 만든다는 표시예요
`hello`	함수 이름이에요. 10의 변수 이름 규칙과 같아요
`()`	재료 칸이에요. 3파트에서 채워요
`:`	콜론이에요. if, for 줄 끝과 같아요
`4칸`	들여쓴 줄이 함수 몸통이에요

모양은 if, for와 같아요. 콜론 뒤 4칸 들여쓴 줄이 한 덩어리예요.

2. 정의와 호출

정의만 하면 아무것도 찍히지 않아요

[1]
```
def hello():
    print("어서 오세요")
    print("주문하시겠어요?")
```
Python

출력 없음

[2]
```
hello()
```
Python

어서 오세요

주문하시겠어요?

hello 이 이름에 몸통을 기억해 둬요

```
print("어서 오세요")
print("주문하시겠어요?")
```

레시피 카드와 같아요

def는 카드에 만드는 법을 적어 두기만 해요. 음료는 주문이 들어올 때, 곧 hello()를 부를 때 만들어요.

def는 기억해 두기만 하고, 실행은 이름()을 부를 때 해요.

2. 정의와 호출

부르면 몸통으로 갔다가 제자리로 돌아와요

[3]
```python
print("가게 문 열어요")
hello()
print("다음 손님")
hello()
```

가게 문 열어요
어서 오세요
주문하시겠어요?
다음 손님
어서 오세요
주문하시겠어요?

1	`print("가게 문 열어요")`	셀 1줄
2	`hello()`	몸통으로 가요
3	`  print("어서 오세요")`	몸통 1줄
4	`  print("주문하시겠어요?")`	몸통 2줄, 돌아와요
5	`print("다음 손님")`	셀 3줄
6	`hello()`	다시 몸통으로
7	`  print(...) 두 줄`	몸통 실행 후 끝

함수를 부를 때마다 몸통을 처음부터 끝까지 실행하고 돌아와요.

3. 매개변수와 인자

괄호 안에 재료 칸을 만들어요

[1]
```python
def greet(name):
    print(f"{name}님, 어서 오세요")

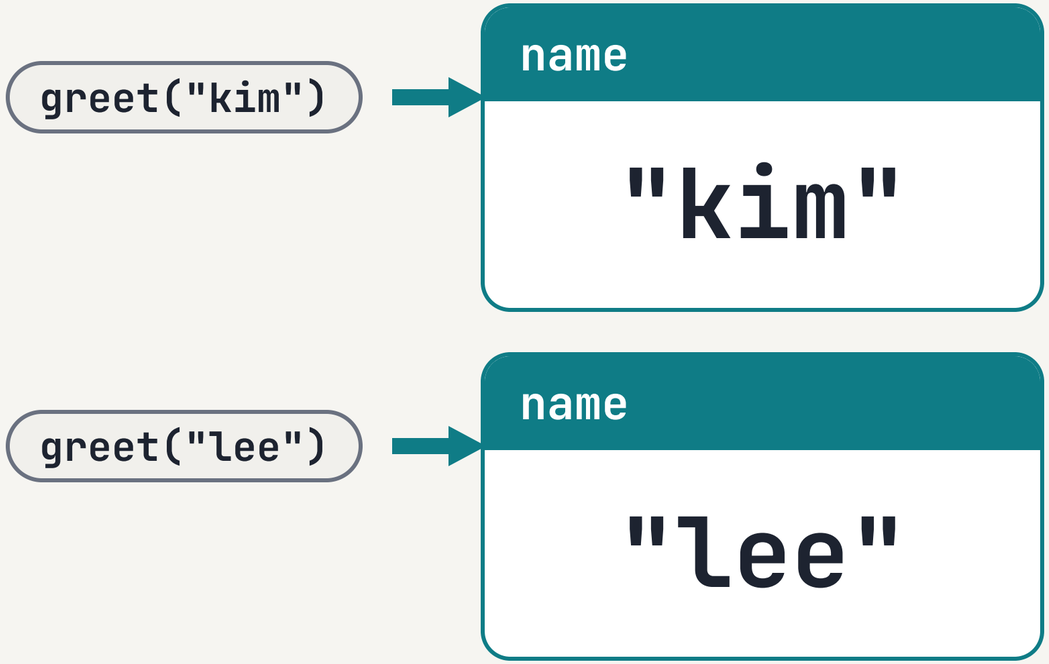
greet("kim")
greet("lee")
```
kim

Python

```
kim님, 어서 오세요
lee님, 어서 오세요
```

name 매개변수. def 괄호 안에 적은 칸 이름이에요

"kim" 인자. 부를 때 괄호에 넣는 값이에요



부를 때 넣은 값이 매개변수 칸에 담겨서 몸통에서 쓰여요.

3. 매개변수와 인자

재료 칸은 쉼표로 여러 개 만들어요

[2]
```python
def order(menu_name, qty):
    print(f"{menu_name} {qty}잔 주문 받았어요")

order("latte", 2)
order("coffee", 1)
```

```
latte 2잔 주문 받았어요
coffee 1잔 주문 받았어요
```

`order("latte", 2)`

menu_name

"latte"

qty

2

첫째는 첫째 칸, 둘째는 둘째 칸

"latte"는 menu_name에, 2는 qty에 들어가요. 적은 순서대로 짝을 지어요.

왜 menu가 아니라 menu_name인가요

15의 menu 딕셔너리와 헷갈리지 않게 이름을 다르게 지었어요. 매개변수 이름도 하는 일이 보이게 지어요.

3. 매개변수와 인자

인자는 적은 순서대로 칸에 들어가요

```
[3]  order(2, "latte")                                    Python
```

```
2 latte잔 주문 받았어요
```

```
order(2, "latte")
```

menu_name	qty
2	"latte"

⚠ 오류 없이 이상하게 찍혀요

파이썬은 값의 뜻을 몰라요. 첫째 값을 첫째 칸에 넣을 뿐이에요. 2가 메뉴 이름이 됐어요.

def 줄을 보고 순서를 맞춰요

order(menu_name, qty)이니 메뉴, 잔 수 순서로 넣어요. 20에서 칸 이름을 적어 넣는 키워드 인자를 배워요.

칸 순서를 바꿔 넣으면 값도 바뀌어 들어가요.

4. return

return은 결과를 부른 자리로 돌려줘요

[1]
```python
def price_of(qty):
    return 4500 * qty                    돌려줄 값

x = price_of(2)
print(x)
print(price_of(3) + 1000)
```
Python

```
9000
14500
```

`x = price_of(2)` → `x = 9000`

`price_of(3) + 1000` → `13500 + 1000`

`13500 + 1000` → `14500`

부른 자리가 값으로 바뀌어요

price_of(2)를 9000으로 바꿔 읽으면 돼요. 변수에 담을 수도 있고, 바로 더하기에 쓸 수도 있어요.

return 뒤에 돌려줄 값을 적어요

return 4500 * qty는 계산한 값을 들고 함수를 나가요. len, sum도 이렇게 값을 돌려줘요.

return 뒤의 값이 함수를 부른 자리에 들어가요.

4. return

print는 화면에 찍기만, return은 값을 돌려줘요

print로 찍는 함수

[2]
```python
def show(qty):
    print(4500 * qty)

a = show(2)
print("a:", a)
```

```
9000
a: None
```

return으로 돌려주는 함수

[3]
```python
def calc(qty):
    return 4500 * qty

b = calc(2)
print("b:", b)
print(b + 1000)
```

```
b: 9000
10000
```

a에 남은 건 None

9000은 화면에 찍혔을 뿐이에요. 함수가 돌려준 값이 없어서 a에는 None만 남았어요.

b에 9000이 담겼어요

돌려받은 값이라 b + 1000처럼 다음 계산에 쓸 수 있어요.

사람이 볼 값은 print, 다음 계산에 쓸 값은 return이에요.

4. return

return이 없으면 None을 돌려줘요

[4]
```python
result = print("안녕")
print(result)
print(type(result))
```

```
안녕
None
<class 'NoneType'>
```

print도 return이 없는 함수예요

print는 화면에 찍는 일만 하고 돌려주는 값이 없어요. 그래서 result에 None이 담겼어요. 앞 장의 show도 같아요.

None은 "값이 없음"이에요

`None`

15에서 get이 없는 키를 만났을 때 돌려준 값과 같아요.

값이 필요하면 return을 적어요

몸통 끝까지 return이 없으면 파이썬이 None을 대신 돌려줘요.

돌려받은 값이 None이면 함수에 return이 있는지 먼저 봐요.

4. return

return을 만나면 함수가 바로 끝나요

[5]
```python
def check(stock):
    if stock == 0:
        return "품절"          # check(0) 여기서 끝
    return "주문 가능"         # check(0)은 안 감

print(check(0))
print(check(3))
```

```
품절
주문 가능
```

부르기	stock == 0	만난 return
check(0)	True	return "품절"
check(3)	False	return "주문 가능"

else를 안 써도 돼요

stock이 0이면 위의 return에서 이미 함수를 나가요. 아래 줄까지 왔다면 0이 아니라는 뜻이에요. 17의 if, else로 써도 결과는 같아요.

return은 값을 돌려주면서 함수를 끝내요. 아래 줄은 실행하지 않아요.

4. return

쉼표로 값 여러 개를 한 번에 돌려줘요

[6]
```python
def summary(prices):
    return sum(prices), len(prices)

print(summary([4500, 5000, 6500]))
total, n = summary([4500, 5000, 6500])
print(total, n)
```

```
(16000, 3)
16000 3
```

받는 쪽은 13의 꺼내 담기

a, b, c = 튜플처럼 total, n = summary(...)로 두 변수에 나눠 담아요. 왼쪽 개수와 돌려준 값 개수가 같아야 해요.

튜플 하나로 돌아와요

`(16000, 3)`

13에서 배운 괄호 묶음, 튜플이에요

total	n
16000	3

return a, b는 튜플로 돌려주고, x, y = 함수()로 나눠 받아요.

5. 오류와 함정

괄호를 빼면 실행하지 않고 함수 자체가 나와요

[1]
```python
def hello():
    print("어서 오세요")

hello
```

```
<function __main__.hello()>
```

[2]
```python
print(hello)
```

```
<function hello at 0x7efc68143530>
```

`hello` → `hello()`

부를 때는 재료가 없어도 괄호 ()를 꼭 붙여요.

어서 오세요가 안 찍혀요

hello만 적으면 "hello라는 함수가 있어요"라고 보여 줄 뿐이에요. 실행하려면 괄호를 붙여요.

0x로 시작하는 숫자는 달라요

함수가 저장된 메모리 위치예요. 실행할 때마다 바뀌어서 화면이 슬라이드와 달라도 괜찮아요.

5. 오류와 함정

칸 수와 인자 수가 다르면 TypeError가 나요

[3]
```python
def order(menu_name, qty):
    print(f"{menu_name} {qty}잔 주문 받았어요")

order("latte")
```

```
TypeError: order() missing 1 required positional argument: 'qty'
```

[4]
```python
order("latte", 2, "L")
```

```
TypeError: order() takes 2 positional arguments but 3 were given
```

missing 1 ... 'qty'

칸 qty가 비었어요. 인자를 하나 덜 넣었어요.

takes 2 ... but 3 were given

칸은 2개인데 인자를 3개 넣었어요.

def 줄의 칸 수만큼 인자를 넣어요.

5. 오류와 함정

return 아래에 적은 줄은 실행되지 않아요

[5]
```python
def calc(qty):
    total = 4500 * qty
    return total                    여기서 나감
    print("계산 끝")                 안 찍힘

print(calc(2))
```

```
9000
```

계산 끝이 안 보여요

return에서 함수를 나갔기 때문이에요. 오류도 나지 않아서 빠진 줄을 알아채기 어려워요.

이렇게 고쳐요 print를 return 위로

```python
def calc(qty):
    total = 4500 * qty
    print("계산 끝")
    return total
```

함수에서 할 일을 다 적고 return은 마지막에 둬요.

5. 오류와 함정

찍기만 하는 함수로 계산하면 TypeError가 나요

[6]
```python
def show(qty):
    print(4500 * qty)

show(2) + 1000
```

```
9000


TypeError: unsupported operand type(s) for +: 'NoneType' and 'int'
```

`show(2) + 1000` → `None + 1000` None에는 더할 수 없어요

9000은 찍혔는데

화면에 찍힌 것일 뿐 돌려준 값은 None이에요. print와 return을 비교한 장의 a와 같아요.

이렇게 고쳐요 print 대신 return

```python
def show(qty):
    return 4500 * qty
```

NoneType이 보이면 return이 빠진 함수를 찾아요.

5. 오류와 함정

정의보다 먼저 부르면 NameError가 나요

[1]
```
welcome()                                  아직 모르는 이름

def welcome():
    print("어서 오세요")
```
Python

```
NameError: name 'welcome' is not defined
```

[2]
```
welcome()
```
Python

```
NameError: name 'welcome' is not defined
```

노트북에서 더 조심해요

def 셀을 지운 뒤에도 커널은 함수를 기억해요. Restart 뒤 위에서부터 실행하면 그제야 오류가 드러나요.

위에서부터 실행해요

첫 줄에서 welcome을 찾았는데 아직 def를 만나지 않았어요. 오류가 나서 아래 def도 실행되지 않았어요.

다시 불러도 같아요

def가 한 번도 실행되지 않았으니까요. def 셀을 먼저 실행하고, 부르는 셀을 아래에 둬요.

6. 실습

장바구니 총액을 함수로 만들어요

[1]
```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500}

def calc_total(cart, menu):
    total = 0
    for m in cart:
        total += menu[m]
    return total

cart1 = ["coffee", "cake"]
cart2 = ["latte", "latte", "cake"]
print(f"합계 {calc_total(cart1, menu):,}원")
print(f"합계 {calc_total(cart2, menu):,}원")
```

1파트의 4줄

손님마다 한 줄

Python

```
합계 11,000원
합계 16,500원
```

복사가 사라졌어요

1파트에서 손님마다 복사한 줄이 함수 안 한 곳에만 있어요.

고칠 곳도 한 곳

할인이 생기면 calc_total만 고치면 돼요.

반복해서 쓰는 계산은 함수로 묶고, return으로 결과를 돌려받아요.

6. 실습

주문할 수 있는지 True, False로 돌려줘요

[2]

```python
def can_order(stock, name):
    if stock[name] > 0:
        return True
    return False

stock = {"coffee": 3, "latte": 0, "cake": 2}
for o in ["coffee", "latte", "cake"]:
    if can_order(stock, o):
        print(o, "주문 받았어요")
    else:
        print(o, "품절이에요")
```

Python

if 뒤에 바로

```
coffee 주문 받았어요
latte 품절이에요
cake 주문 받았어요
```

o	stock[o]	돌려준 값
coffee	3	True
latte	0	False
cake	2	True

이름이 질문이 돼요

if can_order(stock, o):는 "주문할 수 있나요?"로 읽혀요. 조건이 길어져도 함수 안에만 적으면 돼요.

True, False를 돌려주는 함수는 if 뒤에 바로 써요.

6. 실습

17의 등급 계산을 함수로 바꿔요

[3]

```python
def grade(score):
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    return "F"

scores = [["kim", 92], ["lee", 78], ["park", 85], ["choi", 59]]
for name, score in scores:
    print(f"{name:<5}{score:>3}  {grade(score)}")
```

등급만 정해요

Python

```
kim   92  A
lee   78  C
park  85  B
choi  59  F
```

17에서는 반복 안에

```
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
...
print(... grade)
```

반복이 짧아졌어요

for 안에는 찍는 줄 하나만 남았어요.

6. 실습

함수 안에서 다른 함수를 불러요

[4]

```python
def discount_rate(total):
    if total >= 20000:
        return 10
    elif total >= 10000:
        return 5
    return 0

def pay(total):
    rate = discount_rate(total)
    sale = total * rate // 100
    return total - sale

print(pay(8000))
print(pay(15500))
print(pay(22000))
```

다른 함수 부르기

Python

```
8000
14725
19800
```

total	rate	sale	pay
8000	0	0	8000
15500	5	775	14725
22000	10	2200	19800

17의 할인 규칙이에요

할인율 정하기와 결제 금액 계산을 두 함수로 나눴어요. 할인 기준이 바뀌면 discount_rate만 고쳐요.

정리

막히면 오류의 마지막 줄을 봐요

1	**<function hello at 0x...>가 나와요** 괄호를 빼서 실행하지 않았어요	`hello()`
2	**`TypeError: ... missing 1 required positional argument`** 인자를 덜 넣었어요	**def 줄 칸 수 세기**
3	**`TypeError: ... takes 2 positional arguments but 3 were given`** 인자를 더 넣었어요	**def 줄 칸 수 세기**
4	**`NameError: name 'welcome' is not defined`** def 셀보다 먼저 불렀어요	**def 셀 먼저 실행**
5	**돌려받은 값이 None이에요** 함수에 return이 없어요	**print 대신 return**
6	**`TypeError: ... 'NoneType' and 'int'`** None으로 계산했어요	**return 넣기**
7	**return 아래 print가 안 찍혀요** return에서 함수가 끝났어요	**return 위로 옮기기**

결과가 이상하면 함수 안에 print를 넣어 받은 값과 돌려줄 값을 찍어 봐요.

한 장 요약

`def 이름():`

함수 만들기(정의)

`이름()`

함수 실행하기(호출)

`def f(a, b):`

매개변수 칸 두 개

`f(1, 2)`

인자는 순서대로

`return 값`

값을 돌려주고 끝내기

`None`

return이 없을 때

`return a, b`

튜플로 돌려주기

`x, y = f()`

나눠서 받기

사람이 볼 값은 print, 다음 계산에 쓸 값은 return

다음 챕터: 기본값, 키워드 인자, 지역 변수와 전역 변수